

FoRGe: Feedback-Oriented Rule Generation for Security Log Detection

Aidan Wen¹, Karen Shieh¹, Jingzhi Jiang¹, Andy Zhang¹, Raghad Alkhudair²,
Basel Alomair^{2,3,4}, and David Wagner¹

¹ University of California, Berkeley, Berkeley, CA, USA
{aidanwen,karen93,jiangjingzhi,zhangnd16}@berkeley.edu
daw@cs.berkeley.edu

² King Abdulaziz City for Science and Technology (KACST), Riyadh, Saudi Arabia
ralkhudair@kacst.gov.sa
alomair@kacst.edu.sa

³ University of Washington, Seattle, WA, USA

⁴ HUMAIN, Riyadh, Saudi Arabia

Abstract. Manually authoring Security Information and Event Management detection rules is a costly and time-consuming process for security analysts in the modern security environment. To mitigate the burden, we automate the task of example-driven rule synthesis which we aim to instruct large language models (LLMs) to generate accurate regex detection rules given a single malicious log example and optionally a natural language description of the malicious intent. To hold an objective evaluation of our agentic framework, we present a new human-curated dataset that uses endpoint logs from public Splunk security content with 66 independent threat detection tasks as benchmark. We also present FoRGe (Feedback-oriented Rule Generation), an LLM agent which synthesizes regex detection rules, given a malicious log example. It can also take in an optional natural-language description of the malicious activity, as a mechanism for users to guide the agent. FoRGe actively explores available log data in order to craft a future-proof regex that can be used for either retrospective analysis or continuous system protection. We evaluate FoRGe against an independently-designed prompt-only Claude Code baseline across four experimental settings, in both retrospective and continuous monitoring environments, with and without a natural-language description in the role prompts. Our results show that FoRGe achieves a statistically significant advantage in true positive rate over this baseline. Under retrospective analysis, where the entire historical log repository is available to the agents, FoRGe is able to achieve a true positive rate of $93.21\% \pm 3.38\%$ against $79.12\% \pm 6.29\%$ by the Claude agent. When the description is absent, FoRGe demonstrates strong performance, outperforming the Claude agent by $88.48\% \pm 4.72\%$ against $52.34\% \pm 9.35\%$ true positive rate. To isolate the harness from the model, we also evaluate FoRGe against OpenAI’s Codex CLI with both running the same underlying LLM, where FoRGe leads on both true positive rate and false positive rate in every setting. In the continuous monitoring setup, where only the training half of the logs is available to the agents, both agents achieve true positive rates on the unseen test set close to those of the

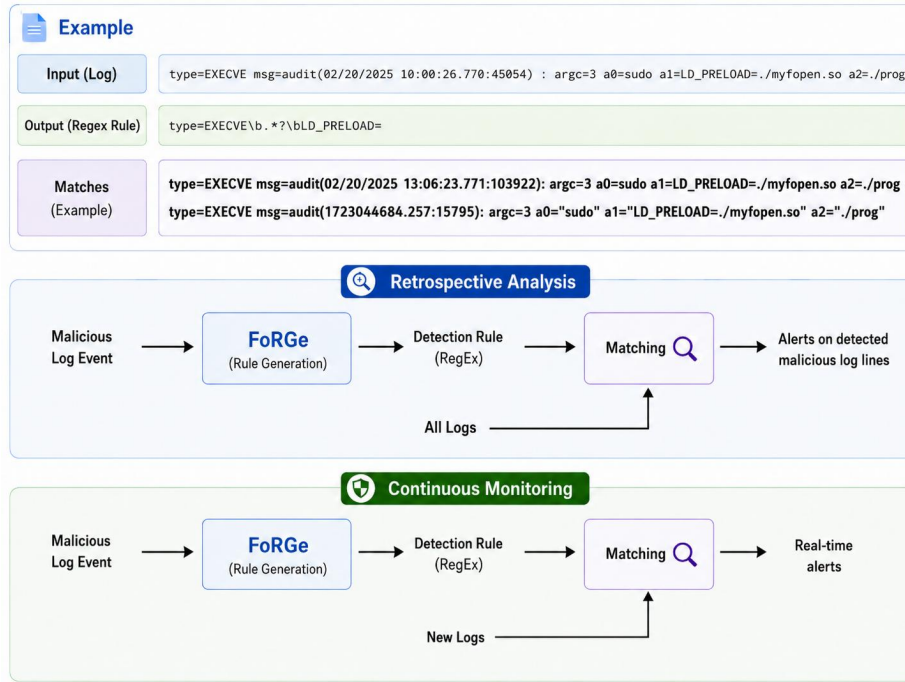


Fig. 1. Overview of FoRGe. **Top:** an example input log line, the regex detection rule FoRGe generates from it, and example log lines matched by the rule. **Middle:** the retrospective analysis setting, in which the generated rule is matched against all historical logs to find past instances of the suspicious behavior. **Bottom:** the continuous monitoring setting, in which the rule is applied to incoming logs to alert on future instances in real time.

retrospective setting. In Claude and Gemini experiments, both agents consistently maintained an average false positive rate of less than 0.5%.

Keywords: Detection Rules · Automated Log Analysis · SIEM Rule Synthesis · Agentic Frameworks

1 Introduction

In modern security operations, analysts detect suspicious behavior using time-stamped records of system events, network activity, and user actions. These are known as security logs. Large systems often generate more log data than any team could read by hand [16], so analysts use queries or patterns that match and flag log entries showing a known suspicious behavior. These are known as detection rules. Writing such rules takes manual effort and time, even when starting with intelligence on the threat. Once written, a rule is static and cheap to run.

In order to assist analysts with this issue, we developed FoRGe (**F**eedback-oriented **R**ule **G**eneration; Figure 1), an LLM agent that automates the process of synthesizing detection rules. We focus on the scenario where a security analyst has noticed a log entry of interest—perhaps it indicates a potential security compromise, for example—and wants to create a detection rule to search for other instances. The analyst provides FoRGe with this example log entry, and optionally a description of what about the log entry is interesting, if the analyst wants to specifically direct the agent on what malicious behavior to focus on in the rule. FoRGe then generates a detection rule—a regex that can be used to search for other instances of similar behavior. FoRGe uses a test-driven agentic loop where the agent tests candidate rules on the provided log corpus and refines them.

We expect FoRGe will be useful in two settings. First, if an analyst identifies a single malicious log entry, they can turn that one example into a set of rules that can be used to search for other such behaviors, or can be used to future-proof the system against the same class of attack by setting up alerts that notify the analyst if similar behavior is observed in the future. Second, if an analyst identifies a single malicious log entry, they can search for similar events in past logs to see if there have been any other prior instances that might not have been caught until now. As regex rules are cheap to run, but still relatively time consuming to construct, the task of constructing them is a good fit for large language model automation. We can run the expensive agent once to produce a rule, then use the regex it produces to efficiently scan logs as they arrive without any further token usage.

In order to effectively evaluate the FoRGe agent, we constructed a human-curated benchmark based on the Splunk security content database, which we call the FoRGe benchmark. It consists of 19,428 total log lines across 66 detection tasks, each task representing a single type of malicious behavior. For each task, we provide labeled positive and negative log lines (positives are examples of malicious behavior, negatives are entirely benign). We hope this benchmark will stimulate and enable future research on this problem.

Our evaluation indicates that the FoRGe agent is effective. Given one example of a certain type of malicious behavior, the rule it crafts detects 85% of other instances of similar behavior in the logs, with a false positive rate around 0.2%–0.4%. If the analyst provides a short description of what about the log message is suspicious, TPR increases above 90% and FPR drops to about 0.1%. Our agent also performs significantly better than a naive baseline (prompting Claude Code).

2 Background and Problem Definition

2.1 Detection Rules

Logs record system events, network traffic, and user activities. They serve as the primary ground truth for detecting anomalies and security breaches.

A detection rule is a saved query or pattern that matches log entries indicating some particular form of suspicious behavior. Rather than reading through terabytes of logs generated daily by large-scale systems [16], analysts write detection rules to identify suspicious log entries for further exploration. However, maintaining an effective library of detection rules requires substantial manual effort from security teams. Engineers must manually translate complex, unstructured threat intelligence into precise, platform-specific queries [12].

Recent advances in LLMs have led researchers to explore their use in automating the generation of detection rules. LLMs can potentially identify suspicious entries in complex log files. However, because of their potential for hallucination [5] and limitations in logical reasoning [3], LLMs are unreliable for this task, and processing entire log files with an LLM is too costly to be operationally practical.

2.2 Task Definition: Rule Synthesis from a Seed Log Message

This paper explores the capability of LLMs to automate the generation of high-quality detection rules, using a custom harness and an agentic feedback loop that enables the model to explore existing log data.

We focus on the following problem. We assume that a human analyst has somehow discovered a log entry that is suspicious, malicious, or otherwise of interest. The AI agent should analyze this log entry to infer what is suspicious about it, then write a rule to match other log entries that are suspicious for the same reason. Optionally, the human analyst can provide a brief description to explain what is suspicious about the log entry, so the agent doesn't have to guess.

We evaluate the generated detection rule using two metrics: true positive rate (TPR), the fraction of suspicious log entries that are detected by the rule (out of all log entries containing the suspicious behavior indicated by the sample log entry), and false positive rate (FPR), the fraction of benign log entries that are detected by the rule (out of all benign log entries). We measure 95% confidence intervals for these two metrics. We do not attempt to measure the agent's ability to match positives that represent a different attack behavior or malicious intent than the input log entry in our evaluation.

Our problem scope is limited: we evaluate each log entry independently in a self-contained fashion. We deliberately exclude detection tasks that require correlation across multiple log entries or aggregation over a time window. In our system, our rules are expressed as a regular expression (regex), but a similar approach could be used to generate Splunk, SQL, or Sigma rules. We chose regex because it is engine-independent and cheap to apply at volume. Translation to native SIEM formats is not evaluated here.

Table 1. Dataset summary statistics.

Metric	Value
Number of tasks	66
Total log lines	19,428
Total positives	501
Total negatives	18,927
Average log lines per task	294.36
Average positives per task	7.59
Average negatives per task	286.77

```

"description": "LD_PRELOAD environment variable used to hook library functions",
"source": {
  "rule_source": "https://github.com/splunk/security_content/blob/develop/detections/endpoint/linux_auditd_preload_hijack_library_calls.yml"
},
"true_positives": [
  "type=EXECVE msg=audit(02/20/2025 13:06:23.771:103922) : argc=3 a0=sudo a1=LD_PRELOAD=./myfopen.so a2=./prog ",
  ...
],
"true_negatives": [
  "type=EXECVE msg=audit(06/05/2025 09:37:00.921:7268) : argc=2 a0=systemctl a1=daemon-reload ",
  ...
]

```

Fig. 2. An example log detection task in the dataset.

3 Design

3.1 The FoRGe Benchmark

We curated a dataset for this task; Table 1 reports the total number of tasks, log lines, true-positive log lines, and true-negative log lines, along with the average number of total, true-positive, and true-negative log lines per task, and Figure 2 illustrates the structure of the dataset. Each task in the dataset contains a selected set of log and a description of the suspicious activity in the true positive log lines.

We constructed our dataset based on threat indicators from the Splunk Attack Range Project [25, 26]. We surveyed existing log rules hand-written in Splunk by expert analysts [27], handpicked 66 of them, and derived one task from each. For each task, we extracted log lines that match the Splunk rule (positives) from Splunk’s attack_data dataset, and found log lines that don’t match any Splunk rule (negatives). For the negatives, we searched especially for log lines that are “hard negatives”, i.e., close in syntax or event type to the positives. We also manually wrote a description for each task and vetted the accuracy of the labels (every true positive should be an instance of the specific suspicious behavior associated with the task, every true negative should be benign) using both AI analysis and manual review. Since many log entries were generated from attack simulations, to prevent label leakage we rename certain hostnames and other keywords that are artifacts of the simulation process (e.g., malware.exe, attack_pc) to innocuous alternatives.

3.2 The FoRGe Agent

We design a feedback-driven harness for crafting detection rules. The harness reasons about what aspects of the input sample are relevant and repeatedly generates a candidate rule, finds all log lines that match the candidate rule, evaluates these matches for relevance (without being given their labels), and refines the rule so it will detect other relevant log entries and avoid matching benign log entries.

Table 2. Agent Tool Surface and Operational Specifications

Tool Signature	Functionality
<code>test_rule(thought, rule)</code>	Executes the regex <code>rule</code> against the input positive. Returns a success flag boolean.
<code>filter_dataset(thought, rule, dataset, [new_dataset])</code>	Runs the parameter <code>rule</code> against the unlabeled main dataset, returning total number of matches and 10 randomly sampled log lines to the agent. The optional <code>new_dataset</code> parameter allows the agent to isolate these matches into a sub-dataset for further exploration.
<code>submit_rule(thought, final_rule)</code>	Submits the finalized <code>final_rule</code> as output and terminates the agentic loop.

To ground the agent’s work, the agent is given three tools (described in Table 2). These tools allow the agent to test/validate a candidate rule: `test_rule` confirms that the candidate regex is syntactically valid and matches the positive provided as input; `filter_dataset` allows the agent to search a large dataset for all entries that match a candidate rule and judge whether the resulting matches would be relevant to an analyst. When the agent is finished with its refinement loop, it calls `submit_rule` to indicate its proposed rule. To ensure that the agent validates its rule candidates, the `submit_rule` tool is not available to the agent until `test_rule` returns `True` and `filter_dataset` has been called at least once, and the prompt instructs the agent to check candidate rules with both tools.

The system prompt defines a structured reasoning process. Initially, the agent identifies relevant fields in the input log that demonstrate the malicious activity. The model is then instructed to brainstorm similar attack methods. Finally, the agent interacts with the tools provided to reduce false positives and uncover attack variants. The prompt is reproduced in condensed form in the Appendix.

To improve the interpretability of the harness, every tool invocation requires a `thought` parameter, which forces the model to explicitly explain its reasoning before taking an action. The harness also produces a structured record that tracks the agent’s strategy throughout the loop. The entire execution history, including all reasoning, tool calling, and responses in the previous steps, remains within the model’s context window throughout the agentic loop.

The agentic loop is limited to 25 turns. As the current loop round number approaches to this limit, the harness includes an alert in the prompt, urging the model to finalize its refinements. While passing `test_rule` and calling

`filter_dataset` typically requires fewer than 25 turns, it encourages the agent to actively explore the dataset and optimize the initial detection draft based on the feedback. On the other hand, imposing an upper bound prevents the model from entering an unbounded loop or overly engineering simple patterns.

3.3 Claude Agent

To compare to a more naive baseline, we also evaluated using Claude Code for this task without any modifications to the harness. This lets us measure the benefit of a custom agent. The Claude agent invokes Anthropic’s Claude Code, with a custom prompt, to generate Python code to implement the detection rule. It is instructed to generate a Python regex based on a single true-positive log entry, then test it on a randomized pool of unlabeled log entries. The prompt defines an *orchestrator* that fans out ten parallel general-purpose sub-agents (one **Agent** tool call each, one per batch) and verifies their outputs, and a *per-agent* contract fixing inputs, a step-by-step procedure, and a strict output schema. The prompt specifies that matches must be produced by `re.search`, not recalled (“do not hallucinate matches”); each regex must match the input true positive before use; and regexes should wildcard incidental fields (timestamps, GUIDs, hosts) to generalize beyond the single example. Both the orchestrator and the ten sub-agents run Claude Opus 4.7 at maximum reasoning effort. The prompt is reproduced in condensed form in the Appendix.

4 Evaluation Implementation

We evaluate the FoRGe agent and Claude agent on our FoRGe benchmark, to measure how reliably each agent can convert an example log line into a detection rule that generalizes to malicious activity without triggering on benign logs. At a high level, to evaluate an agent, we select a true-positive log line from the FoRGe benchmark, run the agent on this positive sample to generate a detection rule, then evaluate the accuracy of the detection rule on the benchmark.

Metrics. We compute a True Positive Rate (TPR) and False Positive Rate (FPR) for each agent, as follows. Each agent accepts a single positive log entry, an unlabelled dataset, and optionally a description, and outputs a detection rule. We calculate the TPR for a single positive by running the agent on that positive along with an unlabelled dataset obtained by taking all other positives from the same task and all negatives from that task, with labels removed and shuffled into a random order, and counting how many other positives from that task are detected by the agent’s detection rule. If a task has n positives, and k of the other positives match when running the agent on one of the positives, then the TPR for that positive is $k/(n - 1)$ (we do not give the agent credit for a detection rule that matches the positive it was provided as input, since that should come automatically, but rather measure generalization to other positives). We calculate the TPR for a task by averaging the TPR of all positives associated

with task (as an implementation detail, if a task has more than 10 positives, we randomly select 10 of them). We calculate the overall TPR of an agent as a macro-average of all of the per-task TPRs. Ten tasks have only one positive, meaning it is not possible to compute their TPR. We thus exclude them from the macro-average. We also exclude one task whose labels were revised after some experiments had completed, since per-rule results for it are not comparable across agents. This leaves 55 tasks used to compute TPR. We compute the FPR similarly over 62 tasks, excluding the same late stage relabeled task, as well as tasks containing no negatives in scope. We repeat the entire process 10 times, average each task’s rate over the repetitions, and report a 95% confidence interval across tasks as $1.96 s/\sqrt{n}$. These intervals capture variability across tasks.

Conditions. We evaluate each agent under two conditions: *with description* provides the agent a short English explanation of what about the input log entry is suspicious, and *without description* does not provide this information. This lets us evaluate two settings, one where the human analyst explains to the agent why the selected log entry is suspicious or relevant to them, and another where the agent tries to figure that out on its own. This lets us evaluate whether agents need assistance from a human analyst about what to focus on, or if they can operate entirely automatically given an example of a suspicious log entry.

Deployment settings. We also evaluate each agent under two deployment settings: *retrospective* and *continuous monitoring*. Retrospective is intended for the usage where an analyst notices a suspicious log entry and then wonders if there are any other past instances of similar suspicious behavior; we want the agent to search all past logs to find any similar log entries, and all log entries can be made available to the log agent to help it craft the detection rule. Continuous monitoring is intended for future use: the analyst notices a suspicious log entry and wants to be notified if any similar events occur in the future; the agent can consult past historical logs, craft a detection rule, and the detection rule will be applied to all future log entries and the analyst notified of any matches. The retrospective setting allows the agent to examine all matches and refine the detection rule as needed, whereas for continuous monitoring the generated detection rule must generalize to future unseen log entries. We simulate a retrospective deployment by providing the agent with an unlabelled dataset containing all log entries. In contrast, we simulate a continuous monitoring deployment with cross-validation: we split the dataset into a training set and test set (50%-50% split), provide the agent with unlabelled entries from the training set, and then evaluate TPR/FPR on the test set. Because some tasks have only one or a few positives, we make sure that the training set has at least one positive for each task.

Agents. We evaluate both the FoRGe agent and Claude agent under all combinations of condition and deployment setting. This allows us to evaluate the benefit of a custom harness vs prompting an existing coding agent. We run FoRGe with Gemini 3.1 Pro Preview and the Claude agent with Claude Opus 4.7 at maximum reasoning effort.

Table 3. Retrospective analysis performance. TPR is averaged over 55 tasks and FPR over 62 tasks; see Section 4 for the excluded tasks.

Description	Agent	Avg. TPR	Avg. FPR
		(Mean $\pm$ 95% CI)	(Mean $\pm$ 95% CI)
With Description	FoRGe	93.21% $\pm$ 3.38%	0.120% $\pm$ 0.104%
	Claude	79.12% $\pm$ 6.29%	0.056% $\pm$ 0.070%
Without Description	FoRGe	88.48% $\pm$ 4.72%	0.186% $\pm$ 0.229%
	Claude	52.34% $\pm$ 9.35%	0.090% $\pm$ 0.158%

5 Results

Tables 3 and 5 present the mean true positive rates and false positive rates along with their respective 95% confidence intervals in four independent experimental settings as described in the previous sections. Across all experiments, FoRGe consistently delivers a significantly higher TPR than Claude agent. Under retrospective analysis with metrics shown in Table 3, FoRGe achieves a TPR of $93.21\% \pm 3.38\%$ and a FPR of $0.120\% \pm 0.104\%$ with both the input log and the natural language description. In comparison, Claude agent yields a lower TPR of $79.12\% \pm 6.29\%$, though it achieves a tighter FPR of $0.056\% \pm 0.070\%$. The non-overlapping confidence intervals confirm that the improvement in TPR is statistically significant. When the description is omitted, performance degrades: FoRGe sees only slight drops, whereas the Claude agent sees a large drop in TPR. This difference highlights FoRGe’s greater capacity to infer malicious intent solely from raw log content.

5.1 Same-Model Control

FoRGe running Gemini 3.1 Pro Preview and the Claude agent running Claude Opus 4.7 at maximum reasoning effort differ in both model and underlying agent, so comparing only these two does not isolate the contribution caused by FoRGe itself. To remedy this, we evaluate FoRGe against OpenAI’s Codex CLI, both running GPT-5.6 Luna at effort=none.

As both harnesses are running the same model, same version, and same effort setting, the harness is the only element that varies.

Opus 4.7 was chosen as a representative of the capable end of models available at the time. Luna was chosen as a representative of weaker and cost effective models.

Both Claude Code and OpenAI’s Codex take in a prompt. This prompt was kept identical between Claude and Codex evaluations, except for references to "Claude Code" and "Codex", which were swapped. Excerpts from the Claude Code prompt are in the appendix.

As shown in Table 4, the FoRGe harness leads on both TPR and FPR in every category. This shows that the advantage FoRGe provides is not specific to

Table 4. Same-model control: retrospective analysis with both harnesses running Luna. This is a control experiment; FoRGe results in Tables 3 and 5 use Gemini 3.1 Pro Preview. TPR is averaged over 55 tasks and FPR over 62 tasks.

Description	Harness	Avg. TPR (Mean $\pm$ 95% CI)	Avg. FPR (Mean $\pm$ 95% CI)
With Description	FoRGe	89.23% $\pm$ 3.36%	0.184% $\pm$ 0.215%
	Codex CLI	70.14% $\pm$ 5.04%	7.635% $\pm$ 1.758%
Without Description	FoRGe	77.01% $\pm$ 7.06%	1.867% $\pm$ 2.312%
	Codex CLI	42.64% $\pm$ 7.72%	3.441% $\pm$ 2.179%

Gemini. Notably, it does not directly confirm how Claude would perform when run through FoRGe.

This also holds true within continuous monitoring and retrospective modes. In retrospective eval mode FoRGe Luna exceeds its corresponding Codex performance by 19.1 TPR with description and 34.4 TPR without. In continuous monitoring mode, FoRGe test set TPR exceeds Codex test set TPR by 14.7 TPR with description and 25.7 TPR without.

In retrospective eval, FoRGe Luna without a description even performs favorably against Codex with one, at 77.01% against 70.14%.

Luna performs notably worse than Gemini and Claude even within FoRGe. FoRGe Luna with description reaches 89.23%, against 93.21% under Gemini 3.1 Pro Preview in Table 3.

Codex Luna’s false positive rate of 7.635% is roughly 136 times the Claude agent’s 0.056%, corresponding to a rule that fires on about one in thirteen benign log lines. FoRGe Luna’s 0.184% with description is higher than the 0.120% it achieves under Gemini, but remains well under half a percent.

Table 5. Continuous monitoring performance on training and test sets. TPR is averaged over 55 tasks and FPR over 62 tasks.

Set	Desc?	Agent	Avg. TPR (Mean $\pm$ 95% CI)	Avg. FPR (Mean $\pm$ 95% CI)
Training	Yes	FoRGe	95.21% $\pm$ 2.75%	0.119% $\pm$ 0.122%
	Yes	Claude	81.26% $\pm$ 5.67%	0.084% $\pm$ 0.111%
	No	FoRGe	87.42% $\pm$ 5.10%	0.138% $\pm$ 0.121%
	No	Claude	52.21% $\pm$ 9.24%	0.348% $\pm$ 0.632%
Test	Yes	FoRGe	92.91% $\pm$ 4.20%	0.125% $\pm$ 0.119%
	Yes	Claude	78.93% $\pm$ 6.88%	0.084% $\pm$ 0.078%
	No	FoRGe	85.76% $\pm$ 6.15%	0.399% $\pm$ 0.531%
	No	Claude	51.94% $\pm$ 9.53%	0.342% $\pm$ 0.632%



Fig. 3. The FoRGe agent achieves better TPR than Claude agent on all settings.

Performance for continuous monitoring is almost as good as retrospective analysis, with a slight decrease in performance (Table 5). Both the FoRGe agent and Claude agent exhibit little overfitting (e.g., 87.42% $\rightarrow$ 85.76%), suggesting that the detection rules generated by the agent appear to generalize well. Figure 3 provides a clear visual comparison of TPRs in the retrospective setting and on the continuous monitoring test set. In the retrospective and continuous monitoring evaluations, both agents consistently maintain low average FPRs, keeping them under 0.5%.

6 Limitations

External validity. The FoRGe benchmark is composed of 66 detection tasks, each of which represents a different type of malicious activity. This is sufficient for a comparative analysis between the FoRGe and Claude agents, but might not be sufficient to predict their performance on rare attacks or enterprise-scale usage. The dataset is built from public Splunk rules and data [25, 27] and is not uniformly sampled from real-world security events, so the distribution of malicious activity or log messages in enterprise deployments might vary. Both true positives and true negatives were generated in a lab environment and did not reflect real operational noise.

Dataset contamination. All benchmark artifacts are public and may appear in the language models’ pretraining data (Gemini 3.1 Pro Preview and Claude Opus 4.7). Consequently, it is possible that both agents might perform better on well-documented attacks in our evaluation while performing worse on unseen, novel attacks.

Confounders. FoRGe and the Claude agent baseline differ in their model capabilities, agent harnesses, tool interfaces, and execution workflows. These differences

may affect the agents’ performance independently of the design choices evaluated in this work. We did not evaluate whether FoRGe’s improvement over Claude can be attributed to our custom harness, a better prompt, or some other difference. The evaluation of FoRGe vs. Codex on Luna shows that the harness contributes independently of the model. However, it does not establish how much of the FoRGe-Gemini advantage over Claude is attributable to our harness rather than to model or some other difference, since Claude was never run on FoRGe.

Label noise. We made our best effort to ensure labels in the FoRGe are accurate, but they could contain errors. We extracted candidate true positives by applying an expert-written detection rule to public log data, and candidate negatives by extracting log entries that did not match any detection rule. We used an LLM to classify every log entry and removed all mislabelled, ambiguous, or borderline log entries. In addition, we used human inspection as an additional cross-check. Excluding borderline log entries could reduce the difficulty of the benchmark.

Scope: single-log detection. The agents are designed to take a single suspicious log line as input and produce one single regex as output. Each log line is ingested as an independent event, to be handled on its own, without taking into account surrounding context. This is a deliberate simplification of the detection problem. In operational detection engineering, many real-world attacks require multi-event correlation or frequency analysis to identify. The agents do not attempt to explore these correlations between distinct events or aggregate over time windows.

Performance at scale. Applying a detection rule to all historical logs may be expensive in the enterprise setting, where data warehouses might store billions of log entries. While matching them against a regex is far faster than processing all of them with an LLM, we did not evaluate the runtime cost of doing so.

Vulnerability. The current system may be vulnerable to data poisoning and adversarial manipulation of log data. For example, attacker can mix benign log with malicious activity log to obscure attack pattern and reduce the likelihood that the resulting behavior is matched by generated log rules. Attacker could also inject benign log that looks similar to attack log to trick agent to discard useful log rules during refinement. We have not evaluated the robustness of our system against such adversarial manipulation, and systematic study on these attacks and potential defense will be important future work.

Comparison with human analysts. Our evaluation measures the quality of generated rules against a labeled benchmark. It does not compare FoRGe against an experienced analyst authoring rules manually. We therefore do not make claims about absolute time saved relative to manual authoring. The monetary cost of running the agents is also left to future work.

7 Related Work

7.1 Detection Rule and Query Generation

Prior work has studied the synthesis of domain-specific language (DSL) programs and rules for data analysis and threat detection, including regex synthesis from examples [14, 21] and natural-language descriptions [20], as well as security signature generation such as Hamsa [15].

More recently, LLMs have been used to support rule generation for security detection. Louro et al. [17] studied the use of LLMs to generate rules for firewalls and intrusion detection systems. LLMCloudHunter [23] uses LLMs to generate generic-signature detection rule candidates from textual and visual Open-source cyber threat intelligence (OSCTI) data. In 2025, Huang et al. [10] built LogRules, a lightweight log analysis framework that generates and utilizes rules through LLMs. LogRules generates natural-language reasoning rules to improve both log parsing and log anomaly detection performance. Contrary to LogRules, SIGMERGE [2] uses LLMs to generate Sigma rules from cyber threat reports (CTRs), targeting log-based security detection rather than general log-analysis reasoning. RulePilot [30] further proposes an LLM-powered agent for security rule generation from natural-language-based descriptions, with support for Splunk SIEM systems. Unlike these works, our setting is example-grounded: an LLM agent receives both an anomalous log, optionally with an activity description, generates a detection rule, and is evaluated against activity-specific labeled logs.

7.2 Log-based Anomaly Detection

Log-based anomaly detection aims to identify system issues or failure through log data. In recent years, people proposed several deep learning methods for log anomaly detection, including supervised approach like LogRobust [33], self-supervised method like LogBERT [9], and unsupervised approach such as DeepLog [6] and LogAnomaly [18]. Recent works also explored the use LLM for log anomaly detection. Qi et al. propose LogGPT [22] which uses ChatGPT for log-based anomaly detection task. Guan et al. [8] use BERT to extract semantic vectors from logs and use Llama for log classifications.

7.3 Log Anomaly Detection Benchmarks

LogHub is one of the most popular datasets for Log Anomaly Detection (LAD) evaluation. LogHub contains a collections of system logs including distributed systems, super computers, operating systems, mobile systems, server applications, and standalone software [34]. Some LogHub datasets originate from prior studies, such as the supercomputer logs studied by Oliner and Stearley [19], while others were collected from real systems in controlled lab environments. LogHub contains both labeled and unlabeled datasets. For example, the Hadoop Distributed File System (HDFS) dataset is labeled and all operating systems

logs are unlabeled. Other newer LAD datasets includes dataset from Hades [13] which contains labeled Apache Spark runtime logs generated in lab environment and the Spirit dataset [19] collected from supercomputer in Sandia National Labs (SNL).

All labeled datasets mentioned above use binary labels that distinguish normal from anomalous behavior. These datasets differ mainly in their labeling granularity. Session-based datasets assign labels to sequences of log messages corresponding to a complete execution unit, such as a block, job, or system instance. In LogHub, HDFS, OpenStack, and Hadoop are commonly treated as session-based datasets. In contrast, log message-based datasets assign labels to individual log messages. Examples include BGL and Thunderbird [19]. Hades [13] uses a different granularity, providing labels for time-window chunks over Apache Spark logs and system metrics.

Prior log anomaly detection datasets contain limited labeled operating-system logs and generally do not categorize anomalous logs by specific suspicious activities. Since our goal is to evaluate an agent’s ability to generate detection rules for particular security-relevant behaviors, existing datasets are insufficient for our setting. Therefore, we construct a new dataset that pairs activity-specific detection tasks with corresponding labeled endpoint logs.

7.4 Agentic Evaluation Harnesses

Recent work on agentic evaluation has moved beyond static question-answering benchmarks and toward harnesses that place models in interactive environments with tools, files, execution feedback, automated scoring, etc. In software engineering, SWE-bench [11] evaluates agents on real GitHub issues by asking them to modify a repository and submit a patch that is checked against tests. SWE-agent [31] further shows that the agent-computer interface (including how the model navigates files, edits code, and runs commands) can strongly affect performance. More recent SWE-bench-style systems, including mini-SWE-agent [28] and SWE-Bench Pro [4], continue this line of work by emphasizing lightweight scaffolds, reproducible execution, and harder long-horizon repository tasks.

Security-focused agentic benchmarks follow a similar pattern but target offensive or risk-oriented tasks rather than general software repair. The NYU CTF Bench [24] and Cybench [32] evaluate agents on capture-the-flag style cybersecurity challenges, where the agent must interact with a controlled environment and recover a flag or complete subtasks. AutoPenBench [7] moves closer to penetration-testing workflows by measuring progress through intermediate milestones. Other benchmarks, such as CyberSecEval 3 [29] and AgentHarm [1], evaluate broader cyber risk and harmful multi-step agent behavior. These works establish the standard practice of evaluating agents inside a reproducible harness, rather than only evaluating the final text produced by a model.

Our setting differs from these benchmarks because the target output is not a code patch, exploit, or CTF flag, but a detection rule that must identify security-relevant behavior in endpoint logs. We therefore adopt an agentic harness design in the spirit of SWE-bench and Cybench: the agent is given a task specification,

access to relevant artifacts, and a controlled evaluation procedure, and its output is scored automatically against labeled log data. This design allows us to evaluate not only whether the model can write a plausible rule, but also whether the generated rule actually detects the intended activity while avoiding unrelated benign logs.

Most existing agentic benchmarks are retrospective: they run agents on a fixed set of tasks and score the final artifact after the run. Our retrospective setting follows this standard practice by evaluating generated rules on previously collected labeled logs. However, security monitoring also requires a continuous setting, where rules are applied to logs as they arrive and must remain useful over time. This split diverges from most current agentic-evaluation practice, where continuous or live evaluation is still uncommon. By separating retrospective evaluation from continuous-monitoring evaluation, we capture both the offline rule-authoring ability emphasized by existing harnesses and the operational requirement that detection rules work in a streaming security-monitoring environment.

8 Ethics and Responsible Disclosure

This work is defensive. We synthesize detection rules (regular expressions that flag malicious behavior in endpoint logs), not exploits or offensive tooling. Our benchmark is built entirely from public Splunk resources: log telemetry from the Splunk Attack Data repository [25], generated in the lab-based Splunk Attack Range [26], with detection tasks adapted from the open-source Splunk Security Content project [27]. All logs are simulated in a sandboxed range, so the dataset contains no personally identifiable information, customer data, or records of real victims, and we conducted no human-subjects research. We release no new attack technique or evasion method, and because we build only on public detections, we uncover no vulnerability requiring disclosure.

9 Artifact and Dataset Availability

We intend to release the artifacts associated with this work to support reproducibility and follow-on research. The release will include: (1) complete log-rule dataset; (2) code for all three agents; (3) evaluation scripts to reproduce experiment results; (4) all experiment data including true positive logs used for rule generation and logs for evaluations.

10 Conclusion and Future Work

In this study, we proposed FoRGe, a framework for using LLMs to generate log-anomaly detection rules based on an example of a problematic log entry. We also constructed a new benchmark for evaluating agents’ ability to generate accurate detection rules. Our experiments show that LLM agents can generate rules that

generalize to similar anomalous log entries while maintaining low false-positive rates. We found that while explaining to the agent why the selected log message is interesting yields the best results, the FoRGe agent remains effective without this guidance, with only a modest drop in true positive rate (93.21% to 88.48% in the retrospective setting). And, our FoRGe agent performed better than a baseline that works by prompting Claude Code.

We see an opportunity to improve on our work by further reducing the false positive rate, extending it to handle complex situations that can only be detected by correlating multiple log events, or achieving better performance without a custom agentic harness.

This work was supported by the KACST-UCB Joint Center on Cybersecurity and Google via the GARA program and a generous gift of Gemini credits. We thank Julien Piet and Alex Kantchelian for helpful guidance.

References

1. Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., Davies, X.: AgentHarm: A benchmark for measuring harmfulness of LLM agents. In: International Conference on Learning Representations (2025), <https://openreview.net/forum?id=AC5n7xHuR1>
2. Cai, Y., Qiu, J., Li, Q., Cheng, D., Chen, L.: From Texts to Rules: Generating Sigma Rules with Large Language Models from Cyber Threat Reports. In: Proceedings of the 35th USENIX Security Symposium. USENIX Association (2026), <https://www.usenix.org/conference/usenixsecurity26/presentation/cai>, prepublication version
3. Cheng, F., Li, H., Liu, F., van Rooij, R., Zhang, K., Lin, Z.: Empowering LLMs with logical reasoning: A comprehensive survey. arXiv preprint arXiv:2502.15652 (2025). <https://doi.org/10.48550/arxiv.2502.15652>
4. Deng, X., Da, J., Pan, E., He, Y.Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., Sampath, K., Krishnan, M., Kundurthy, S., Hendryx, S., Wang, Z., Zhang, C.B.C., Jacobson, N., Liu, B., Kenstler, B.: SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv preprint arXiv:2509.16941 (2025), <https://arxiv.org/abs/2509.16941>
5. Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., Weston, J.: Chain-of-verification reduces hallucination in large language models. arXiv preprint arXiv:2309.11495 (2023). <https://doi.org/10.48550/arxiv.2309.11495>
6. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1285–1298. CCS '17, Association for Computing Machinery, New York, NY, USA (Oct 2017). <https://doi.org/10.1145/3133956.3134015>, <https://dl.acm.org/doi/10.1145/3133956.3134015>
7. Gioacchini, L., Delsanto, A., Drago, I., Mellia, M., Siracusano, G., Bifulco, R.: AutoPenBench: A vulnerability testing benchmark for generative agents. In: Potdar, S., Rojas-Barahona, L., Montella, S. (eds.) Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track. pp. 1615–1624. Association for Computational Linguistics, Suzhou, China

- (Nov 2025). <https://doi.org/10.18653/v1/2025.emnlp-industry.114>, <https://aclanthology.org/2025.emnlp-industry.114/>
8. Guan, W., Cao, J., Qian, S., Gao, J., Ouyang, C.: LogLLM: Log-based Anomaly Detection Using Large Language Models (Apr 2025). <https://doi.org/10.48550/arXiv.2411.08561>, <http://arxiv.org/abs/2411.08561>, arXiv:2411.08561 [cs.SE]
 9. Guo, H., Yuan, S., Wu, X.: LogBERT: Log Anomaly Detection via BERT. In: 2021 International Joint Conference on Neural Networks (IJCNN). pp. 1–8 (Jul 2021). <https://doi.org/10.1109/IJCNN52387.2021.9534113>, <https://ieeexplore.ieee.org/document/9534113/>, ISSN: 2161-4407
 10. Huang, X., Zhang, T., Zhao, W.: LogRules: Enhancing Log Analysis Capability of Large Language Models through Rules. In: Chiruzzo, L., Ritter, A., Wang, L. (eds.) Findings of the Association for Computational Linguistics: NAACL 2025. pp. 452–470. Association for Computational Linguistics, Albuquerque, New Mexico (Apr 2025). <https://doi.org/10.18653/v1/2025.findings-naacl.28>, <https://aclanthology.org/2025.findings-naacl.28/>
 11. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world GitHub issues? In: International Conference on Learning Representations (2024), <https://openreview.net/forum?id=VTF8yNQm66>
 12. Konstantaras, I., Chatzoglou, E., Kampourakis, K.E., Kambourakis, G.: Evaluating llms for the automated generation of operational detection rules in enterprise edr environments. *Electronics* **15**(10) (2026). <https://doi.org/10.3390/electronics15102088>, <https://www.mdpi.com/2079-9292/15/10/2088>
 13. Lee, C., Yang, T., Chen, Z., Su, Y., Yang, Y., Lyu, M.R.: Heterogeneous Anomaly Detection for Software Systems via Semi-Supervised Cross-Modal Attention. In: Proceedings of the 45th International Conference on Software Engineering. pp. 1724–1736. ICSE '23, IEEE Press, Melbourne, Victoria, Australia (Jul 2023). <https://doi.org/10.1109/ICSE48619.2023.00148>, <https://dl.acm.org/doi/10.1109/ICSE48619.2023.00148>
 14. Lee, M., So, S., Oh, H.: Synthesizing regular expressions from examples for introductory automata assignments. *SIGPLAN Not.* **52**(3), 70–80 (Oct 2016). <https://doi.org/10.1145/3093335.2993244>, <https://doi.org/10.1145/3093335.2993244>
 15. Li, Z., Sanghi, M., Chen, Y., Kao, M.Y., Chavez, B.: Hamsa: Fast signature generation for zero-day polymorphic worms with provable attack resilience. In: 2006 IEEE Symposium on Security and Privacy. pp. 32–47 (2006). <https://doi.org/10.1109/SP.2006.18>
 16. Locke, S., Li, H., Chen, T.H.P., Shang, W., Liu, W.: Logassist: Assisting log analysis through log summarization. *IEEE Transactions on Software Engineering* **48**(9), 3227–3241 (2022). <https://doi.org/10.1109/TSE.2021.3083715>
 17. Louro, B., Abreu, R., Cabral Costa, J., F. Sequeiros, J.B., M. Inácio, P.R.: Analysis of the Capability and Training of Chat Bots in the Generation of Rules for Firewall or Intrusion Detection Systems. In: Proceedings of the 19th International Conference on Availability, Reliability and Security. pp. 1–7. ARES '24, Association for Computing Machinery, New York, NY, USA (Jul 2024). <https://doi.org/10.1145/3664476.3670902>, <https://doi.org/10.1145/3664476.3670902>
 18. Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., Zhou, R.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19. pp.

- 4739–4745. International Joint Conferences on Artificial Intelligence Organization (Jul 2019). <https://doi.org/10.24963/ijcai.2019/658>, <https://doi.org/10.24963/ijcai.2019/658>
19. Oliner, A., Stearley, J.: What Supercomputers Say: A Study of Five System Logs. In: Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. pp. 575–584. DSN '07, IEEE Computer Society, USA (Jun 2007). <https://doi.org/10.1109/DSN.2007.103>, <https://doi.org/10.1109/DSN.2007.103>
 20. Park, J.U., Ko, S.K., Cognetta, M., Han, Y.S.: SoftRegex: Generating regex from natural language descriptions using softened regex equivalence. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). pp. 6425–6431. Association for Computational Linguistics, Hong Kong, China (Nov 2019). <https://doi.org/10.18653/v1/D19-1677>, <https://aclanthology.org/D19-1677/>
 21. Pertseva, E., Barbone, M., Rudek, J., Polikarpova, N.: Regex+: Synthesizing regular expressions from positive examples. In: 11th Workshop on Synthesis (2022), <https://par.nsf.gov/biblio/10336574>
 22. Qi, J., Huang, S., Luan, Z., Yang, S., Fung, C., Yang, H., Qian, D., Shang, J., Xiao, Z., Wu, Z.: LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In: 2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys). pp. 273–280 (Dec 2023). <https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045>, <https://ieeexplore.ieee.org/document/10466820/>
 23. Schwartz, Y., Ben-Shimol, L., Mimran, D., Elovici, Y., Shabtai, A.: LLMCloud-Hunter: Harnessing LLMs for Automated Extraction of Detection Rules from Cloud-Based CTI. In: Proceedings of the ACM on Web Conference 2025. pp. 1922–1941. WWW '25, Association for Computing Machinery, New York, NY, USA (Apr 2025). <https://doi.org/10.1145/3696410.3714798>, <https://dl.acm.org/doi/10.1145/3696410.3714798>
 24. Shao, M., Jancheska, S., Udeshi, M., Dolan-Gavitt, B., Xi, H., Milner, K., Chen, B., Yin, M., Garg, S., Krishnamurthy, P., Khorrami, F., Karri, R., Shafique, M.: NYU CTF bench: a scalable open-source benchmark dataset for evaluating LLMs in offensive security. In: Proceedings of the 38th International Conference on Neural Information Processing Systems. NIPS '24, Curran Associates Inc., Red Hook, NY, USA (2024)
 25. Splunk Inc.: Splunk Attack Data. https://github.com/splunk/attack_data (2025), a repository of curated datasets from various attacks. Accessed: 2026-06-04
 26. Splunk Inc.: Splunk Attack Range v5. https://github.com/splunk/attack_range (2026), accessed: 2026-06-04
 27. Splunk Inc.: Splunk Security Content. https://github.com/splunk/security_content (2026), accessed: 2026-06-04
 28. SWE-agent Team: mini-SWE-agent: The 100 line AI agent that solves GitHub issues or helps you in your command line. <https://github.com/SWE-agent/mini-swe-agent> (2025), gitHub repository
 29. Wan, S., Nikolaidis, C., Song, D., Molnar, D., Crnkovich, J., Grace, J., Bhatt, M., Chennabasappa, S., Whitman, S., Ding, S., Ionescu, V., Li, Y., Saxe, J.: CYBERSECEVAL 3: Advancing the evaluation of cybersecurity risks and capabil-

- ities in large language models. arXiv preprint arXiv:2408.01605 (2024), <https://arxiv.org/abs/2408.01605>
30. Wang, H., Xu, M., Guo, Y., Han, W., Lim, H.W., Dong, J.S.: RulePilot: An LLM-powered agent for security rule generation. In: Proceedings of the 48th International Conference on Software Engineering (2026), <https://conf.researchr.org/details/icse-2026/icse-2026-research-track/207/RulePilot-An-LLM-Powered-Agent-for-Security-Rule-Generation>
 31. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-computer interfaces enable automated software engineering. In: Advances in Neural Information Processing Systems (2024), https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf
 32. Zhang, A.K., Perry, N., Dulepet, R., Ji, J., Menders, C., Lin, J.W., Jones, E., Hussein, G., Liu, S., Jasper, D., Peetathawatchai, P., Glenn, A., Sivashankar, V., Zamoshchin, D., Glikbarg, L., Askaryar, D., Yang, M., Zhang, T., Alluri, R., Tran, N., Sangpisit, R., Yiorkadjis, P., Osele, K., Raghupathi, G., Boneh, D., Ho, D.E., Liang, P.: Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In: International Conference on Learning Representations (2025), <https://openreview.net/forum?id=tc90LV0yRL>
 33. Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.G., Chintalapati, M., Shen, F., Zhang, D.: Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 807–817. ESEC/FSE 2019, Association for Computing Machinery, New York, NY, USA (Aug 2019). <https://doi.org/10.1145/3338906.3338931>, <https://dl.acm.org/doi/10.1145/3338906.3338931>
 34. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A large collection of system log datasets for ai-driven log analytics. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). pp. 355–366 (2023). <https://doi.org/10.1109/ISSRE59848.2023.00071>

Appendix

For reproducibility, this appendix presents the core prompts of both agents in condensed form: orchestration wrapping, exact file paths, verbatim output schemas, and some lengthy sub-instructions (marked as omitted in the listings) are excluded for brevity.

FoRGe Prompt

```
# ROLE
You are an expert Detection Engineer. Your goal is to write a robust
detection rule for the provided suspicious log using a Python-compatible
regular expression string.

# REASONING PROTOCOL
You MUST provide your step-by-step reasoning in a text block before every
tool call. Use the following Markdown headers for your reasoning:

### 1. Log Analysis
Deconstruct the log. Identify static signatures (API names, specific
flags) vs. transient data (IPs, hostnames, GUIDs).

### 2. Variation Brainstorming
Based on the 'description', how else might this attack be executed?
- List 2-3 potential variations (e.g., different command flags,
synonymous paths, alternative tool aliases, or different event types/IDs
that log the same behavior).
- Your rule SHOULD account for these variations to ensure high recall.
- Also list 1-2 **legitimate scenarios** that look superficially similar
(e.g., admin tools, system accounts, known software installers). Your
rule MUST exclude or not match these.

### 3. Detection Strategy
Explain your plan. How will you match the behavior intent while avoiding
common false positives? Why is this approach better than a simple literal
string match?

### 4. Historical Review
Analyze the output of your PREVIOUS tool call (if any).
- **If Turn 1**: State "No previous tests have been run."
- **If previous test failed**: Explain the exact error and how your new
draft fixes it.
- **If previous test passed**: Evaluate if it is "good enough" or needs
refinement to be more robust.

### 5. Drafting
Provide the draft of your regex code.
```

```

# DETECTION PRINCIPLES
1. **Generalize Intent**: The input log is just ONE example. Use the ‘
description’ to understand the malicious intent and write a rule that
catches the *behavior*, not just this specific string instance.
2. **Robust Patterns**: <Prompts for generating robust regex omitted>
<Additional instructions for rule generation omitted>

# REFINEMENT PROTOCOL
<Prompts for refining rule omitted>

# DATASET DISCOVERY AND ITERATION
You have access to a ‘filter_dataset’ tool that allows you to apply your
detection rule or any search query to a larger dataset (e.g., the “main
” dataset).
- **Mixed Environment**: The “main” dataset contains a **mixture of
both malicious and benign logs** related to the current behavior. You are
NOT told which is which.
- **Match Limit Warning**: ‘filter_dataset’ returns a MAXIMUM of 10
random matches. If a broad query returns hundreds of matches, the
malicious logs might be hidden by benign noise in your 10-log sample. If
this happens, progressively narrow your query to surface the hidden
malicious variants.
- **Reduce False Positives**: After passing ‘test_rule’ for your initial
log, use ‘filter_dataset’ on “main” with your candidate rule to see
what logs it catches. <Additional instructions on using the tools omitted
>

# ORDER OF OPERATIONS
- ‘test_rule’ is your primary tool for verifying your rule against the
specific input example.
- ‘filter_dataset’ MUST be used after a successful ‘test_rule’ to find
False Positives with your rule, and to find False Negatives using a
broader search query, before you ‘submit_rule’.
- ‘submit_rule’ is LOCKED until you have received ‘success: True’ from ‘
test_rule’ in a tool response.

Suspicious Log Line (EXAMPLE):
“”
<Input log line>
“”

# MANDATORY INSTRUCTIONS
1. Follow the REASONING PROTOCOL using the required Markdown headers. **
Variation Brainstorming is critical.**
2. The provided log is just ONE possible instance. Your goal is to
achieve HIGH RECALL by detecting the underlying behavior described.
3. You are currently in the INITIAL state. You MUST call ‘test_rule’
first to verify the input log.

```

4. **Discovery Phase**: You MUST use 'filter_dataset' against the "main" dataset. First, test your rule to find and eliminate **False Positives**. Second, test a deliberately broad keyword search (e.g., just the executable name) to discover hidden attack variants and reduce **False Negatives**.

Claude Agent Prompt

You are processing a batch of detection tasks. For each input:

INPUTS

- true_positive: one log line -- your ONLY signal about what the rule should detect. No description; the filename carries no semantic hint.
- randomized pool: a JSON array of logs mixing true positives and true negatives. Labels are NOT exposed.

TASK (for each input)

1. From the log alone, infer the likely malicious indicator(s) -- suspicious binary names, command-line arguments, registry paths, network destinations, event IDs, etc.
2. Craft a Python regex capturing that indicator; `re.search(regex, true_positive)` must succeed -- revise until it does.
3. Run `re.search(regex, log)` against EVERY log in the pool and collect the matches.
4. LLM-judged refinement loop: read each matched log and judge whether it shows the malicious behavior the true_positive exemplifies. If any match is benign, the regex is too permissive -- tighten it so it still matches the true_positive but not the benign log(s), then re-scan and re-judge. Repeat until all matches are judged malicious, or 25 iterations. If you cannot tell, treat the match as a false positive and tighten.

Do NOT hallucinate matches: every log you report as a match must be a literal `re.search` hit with the final regex.

CONSTRAINTS

- Use Python's `re` module; `re.DOTALL` for XML/multiline logs.
- Be specific enough to avoid obvious false positives, but wildcard incidental fields (timestamps, GUIDs, host names) so the regex generalizes beyond the single example.

OUTPUT (for each input): the final regex, the pool logs it matched, the number of refinement iterations, and the benign logs excluded during tightening.

A Example Generated Rules

This section shows examples of rules FoRGe produces. The agent ingests a positive log line and an optional short description, and outputs one regex. This is then scored on the remaining positives and negatives of that task. Below are examples of the regexes produced along with basic information about that regex. Note that these regexes are examples specifically shown to illustrate rule structure, not randomly selected.

Both examples are from the FoRGe (Gemini 3.1 Pro) retrospective condition, from the runs reported in Table 3.

Example 1. Detects procdump or a variant executing with flags that dump LSASS process memory.

<code>(?i)-m[amp]\s+[']?lsass\b</code>	
Description given	none
Seed	2,098 bytes, Sysmon Event ID 1 (XML)
Regex length	26 characters
True positives	31 / 31 held-out positives
False positives	0 / 270 negatives
Execution time	0.39 ms over the rule's corpus
Retained from seed	-ma (widened to -m[amp]), lsass
Matched positive	2,344 bytes, CrowdStrike JSON; matched on -mm lsass

```
Seed (2,098 bytes, truncated):
<Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System
  ><Provider Name='Microsoft-Windows-Sysmon' Guid='{5770385F-C22A-43E0-BF4C
    -06F5698FFBD9}'><EventID>1</EventID><Version>5</Version><Level>4</Level
    ><Task>1</Task><Opcode>0</Opc
... (truncated) ...
\Windows\system32\cmd.exe" /c "C:\internal_ops\scripts\REF_ID_4333\bin\
  procdump.exe -accepteula -ma lsass.exe C:\Windows\Temp\lsass_dump.dmp" </
  Data><Data Name='CurrentDirectory'>C:\Users\ADMINI~1\AppData
... (truncated)

Matched held-out positive (2,344 bytes, truncated):
{"WindowTitle":"C:\\internal_ops\\scripts\\REF_ID_4333\\bin\\procdump64.exe
  ","ProcessCreateFlags":"0","IntegrityLevel":"12288","ParentProcessId
    ":"285474566","SourceProcessId":"285474566","aip":"35.157.24.242","
    SHA1HashData":"00000000000000000000000000000000
... (truncated) ...
015406.1","CommandLine":"C:\\internal_ops\\scripts\\REF_ID_4333\\bin\\
  procdump.exe -accepteula -mm lsass.exe C:\\Windows\\Temp\\lsass_dump.dmp
    ","ParentAuthenticationId":"1657682","TargetProcessId":"2
... (truncated)
```

Example 2. Detects PowerShell reflectively loading .NET assembly payloads.

```
(?i)\\[s*(?:System\\.)?Reflection\\.Assembly\s*\\]::Load\b
```

Description given	“PowerShell reflectively loading .NET assembly payloads.”
Seed	187 bytes, PowerShell command line
Regex length	55 characters
True positives	18 / 18 held-out positives
False positives	0 / 191 negatives
Execution time	0.11 ms over the rule’s corpus
Retained from seed	[Reflection.Assembly]::Load; System. prefix made optional, though absent from the seed
Matched positive	2,545 bytes, PowerShell Event ID 4104 (XML); matched on [System.Reflection.Assembly]::Load

```
Seed (187 bytes):
[Reflection.Assembly]::Load([System.Convert]::FromBase64String((New-Object
    Net.WebClient).DownloadString('https://pastebin.com/raw/d46Ek57')).
    replace('%$','$A'))).EntryPoint.Invoke($N,$N)

Matched held-out positive (2,545 bytes, truncated):
<Event xmlns='http://schemas.microsoft.com/win/2004/08/events/event'><System
    ><Provider Name='Microsoft-Windows-PowerShell' Guid='{A0C1853B-5C40-4B15
    -8766-3CF1C58F985A}'/><EventID>4104</EventID><Version>1</Version><Level
    >3</Level><Task>2</Task><Opcode
... (truncated) ...
1</Data><Data Name='MessageTotal'>1</Data><Data Name='ScriptBlockText'>
    $assembly = [System.Reflection.Assembly]::Load((Invoke-WebRequest 'http
    ://127.0.0.1/malicious-binary.exe' -UseBasicParsing).Conte
... (truncated)
```

B Description Length Statistics

In the with-description condition, each task is accompanied by a short natural-language description of the suspicious behaviour. Table 6 reports the length of these descriptions in words. The descriptions are brief: the longest is 15 words and the median is 8. The intent is guidance as opposed to direct instruction.

Table 6. Length of the rule descriptions given to the agents in the with-description condition, in words (whitespace-separated tokens), over all 66 tasks in the FoRGe benchmark.

Statistic Words	
Mean	7.68
Median	8
Max	15
Min	2